Furqan Memory Skill — Product Documentation

Complete Technical Reference

Project: Furqan Memory — Client-side memory and learning for AI agents
Version: 0.1.0
Date: March 21, 2026
Location: `furqan-memory/`
Protocol: JSON-RPC 2.0 over stdio (MCP-compatible)

Table of Contents

---

1. What is Furqan Memory?

Furqan Memory gives AI agents **persistent memory across sessions**. It stores evaluation verdicts locally, learns patterns from repeated queries, and enables fast-path recognition for known question types.

Key Features

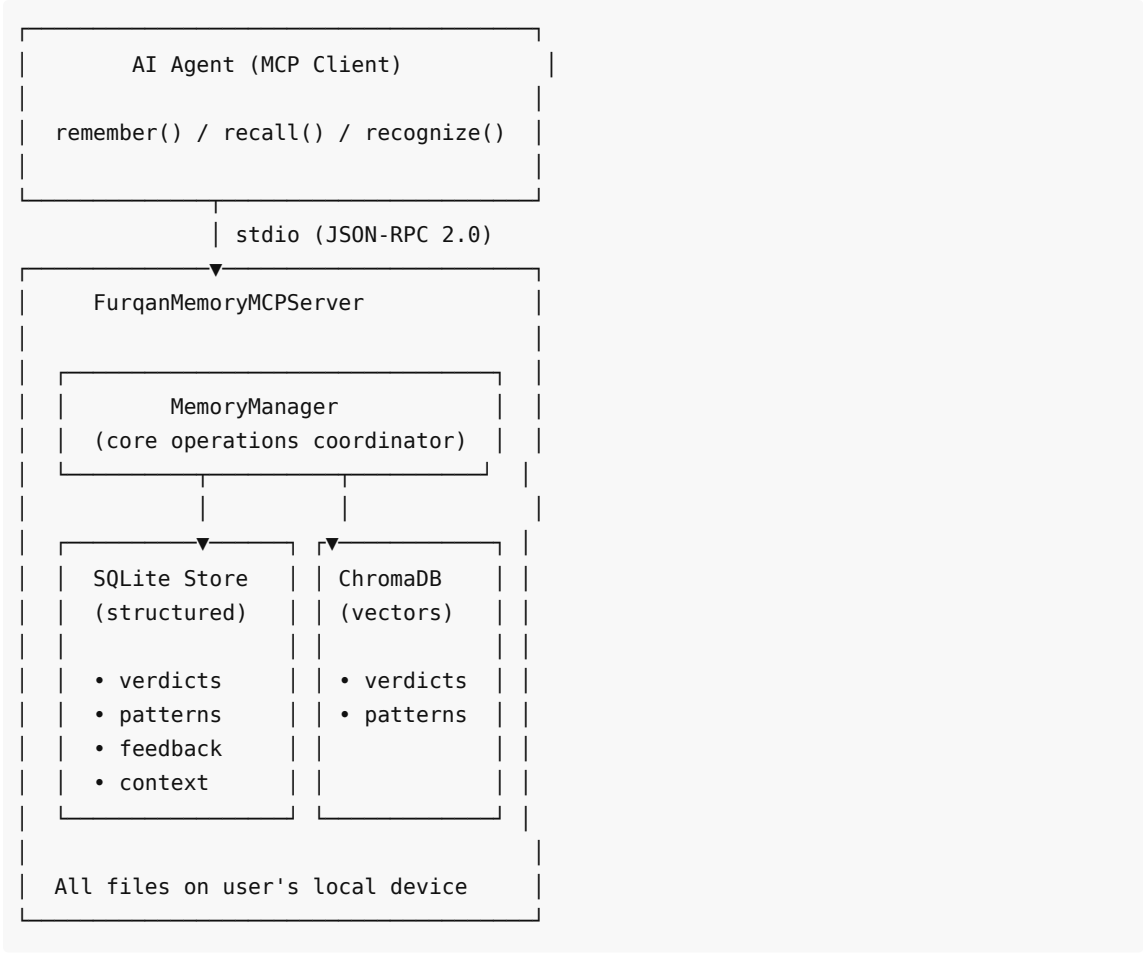
- **Remember:** Store verdicts after evaluation
- **Recall:** Semantic search over past verdicts
- **Recognize:** Fast-path pattern matching (<50ms target)
- **Feedback:** Rate verdicts to improve pattern accuracy
- **Stats:** Memory usage and pattern maturity tracking

Why Client-Side Only?

- **Privacy:** User's questions and verdicts never leave their device
- **No network dependency:** Works offline, no cloud sync
- **No telemetry:** Zero data collection
- **User ownership:** Delete the DB file = all data gone

---

2. Architecture



---

3. Package Structure

```
furqan-memory/
├── pyproject.toml                      # Package metadata
├── README.md                           # Quick start guide
├── SKILL.md                            # MCP skill manifest
├── src/
│   └── furqan_memory/
│       ├── __init__.py
│       ├── __main__.py                 # Entry point: python -m furqan_memory
│       ├── mcp_server.py               # MCP server (5 tools)
│       ├── memory_manager.py           # Core operations coordinator
│       └── storage/
│           ├── __init__.py
│           ├── sqlite_store.py         # Structured storage (4 tables)
│           └── vector_store.py         # ChromaDB semantic search
└── tests/
    ├── __init__.py
    ├── test_mcp_memory.py              # 16 MCP server tests
    ├── test_memory_manager.py          # 14 manager tests
    ├── test_sqlite_store.py            # 18 SQLite tests
    └── test_vector_search.py           # 8 vector tests
```

---

4. SQLite Schema

Database file: `furqan_memory.db` (configurable via `FURQAN_MEMORY_DB` env var)

Table 1: `verdicts`

Stores past evaluation results.

Column	Type	Description
`id`	TEXT PK	Unique verdict ID (e.g., `v_abc123def456`)
`question`	TEXT NOT NULL	The original question
`domain`	TEXT	Knowledge domain (default: `'islamic'`)
`verdict_json`	TEXT NOT NULL	Full verdict data as JSON
`total_score`	INTEGER	Overall score (0-100)
`gate_results`	TEXT	Gate scores as JSON array
`final_judgment`	TEXT	Final judgment text
`tags`	TEXT	JSON array of tags (default: `'[]'`)
`created_at`	REAL NOT NULL	Unix timestamp
`accessed_at`	REAL	Last access timestamp
`access_count`	INTEGER	Number of times accessed (default: 0)

Table 2: `patterns`

Learned reasoning patterns that enable fast-path recognition.

Column	Type	Description
`id`	TEXT PK	Pattern ID (e.g., `p_abc123def456`)
`category`	TEXT NOT NULL	Pattern category (e.g., `"riba"`, `"general"`)
`domain`	TEXT	Knowledge domain (default: `'islamic'`)
`rule`	TEXT NOT NULL	Pattern rule description
`signals`	TEXT NOT NULL	JSON array of signal keywords
`expected_gates`	TEXT NOT NULL	Expected gate results as JSON
`expected_score_min`	INTEGER	Minimum expected score
`expected_score_max`	INTEGER	Maximum expected score
`template`	TEXT	Response template
`confidence`	REAL	Pattern confidence (0.0-1.0, default: 0.3)

`source_verdicts`	TEXT	JSON array of source verdict IDs (default: `'[]'`)
`hit_count`	INTEGER	Times this pattern was matched (default: 0)
`last_hit`	REAL	Last time pattern was matched
`created_at`	REAL NOT NULL	Creation timestamp
`feedback_score`	REAL	Cumulative feedback score (default: 0.0)
`version`	INTEGER	Pattern version (default: 1)

Table 3: `feedback`

User feedback on verdicts and patterns.

Column	Type	Description
`id`	TEXT PK	Feedback ID (e.g., `f_abc123def456`)
`target_id`	TEXT NOT NULL	The verdict or pattern being rated
`target_type`	TEXT NOT NULL	`"verdict"` or `"pattern"`
`rating`	TEXT NOT NULL	`"positive"`, `"negative"`, or `"neutral"`
`correction`	TEXT	Optional correction text
`created_at`	REAL NOT NULL	Timestamp

Table 4: `context`

Key-value context storage for agent preferences.

Column	Type	Description
`key`	TEXT PK	Context key
`value`	TEXT NOT NULL	JSON-encoded value
`domain`	TEXT	Scope domain (default: `'global'`)
`updated_at`	REAL NOT NULL	Last update timestamp

---

5. ChromaDB Vector Search

File: `storage/vector_store.py`

Class: `MemoryVectorSearch`

Two separate ChromaDB collections:

Collection	Document	Purpose
`memory_verdicts`	Question text	Semantic search over past verdicts

`memory_patterns`	Pattern rule text	Fast-path pattern recognition

Both use **cosine distance** (`hnsw:space: cosine`).

Key Methods

```
class MemoryVectorSearch:
    def __init__(self, persist_dir: str | None = None):
        """None = ephemeral (for tests). Path = persistent."""

    def add_verdict(self, id: str, question: str, metadata: dict = None)
    def add_pattern(self, id: str, rule: str, metadata: dict = None)
    def search_verdicts(self, query: str, limit: int = 5) -> list[dict]
    def search_patterns(self, query: str, limit: int = 5) -> list[dict]
```

Score Conversion

ChromaDB returns **distances** (lower = more similar for cosine). Converted to similarity scores:

```
score = max(0.0, 1.0 - distance)  # 1.0 = identical, 0.0 = completely different
```

Metadata Safety

ChromaDB only accepts `str` , `int` , `float` , `bool` as metadata values. The `_safe_metadata()` method automatically converts complex types to JSON strings and drops `None` values.

---

6. MemoryManager

File: `memory_manager.py`

Coordinates SQLite (structured) and ChromaDB (vector) storage.

`remember(question, verdict, domain, tags) → verdict_id`

Stores a verdict in both SQLite and ChromaDB:

```
def remember(self, question, verdict, domain="islamic", tags=None):
    verdict_id = f"v_{uuid4().hex[:12]}"
    # 1. Save to SQLite (structured data)
    self.store.save_verdict(verdict_id, question, verdict, domain, tags)
    # 2. Add to ChromaDB (vector index for semantic search)
    self.vectors.add_verdict(verdict_id, question, metadata={
        "domain": domain,
        "total_score": verdict.get("total_score", 0),
        "final_judgment": verdict.get("final_judgment", ""),
    })
    return verdict_id
```

`recall(query, domain, limit) → list[dict]`

Semantic search → enrichment:

```
def recall(self, query, domain="all", limit=5):
    # 1. Vector search for similar questions
    vector_results = self.vectors.search_verdicts(query, limit)
    # 2. Enrich with full data from SQLite
    for vr in vector_results:
        verdict = self.store.get_verdict(vr["id"])
        # Filter by domain, build enriched result
    return enriched
```

`recognize(query, threshold) → dict | None`

Fast-path pattern matching. Target: **<50ms**.

```
def recognize(self, query, threshold=0.75):
    matches = self.vectors.search_patterns(query, limit=1)
    if not matches or matches[0]["score"] < threshold:
        return None
    pattern = self.store.get_pattern(matches[0]["id"])
    if not pattern or pattern["confidence"] < 0.8:
        return None
    # Update hit tracking
    self.store.update_pattern(matches[0]["id"], {
        "hit_count": pattern["hit_count"] + 1,
        "last_hit": time.time(),
    })
    return {"matched": True, "pattern": pattern, "similarity": matches[0]["score"]}
```

Recognition criteria:

1. Vector similarity ≥ `threshold` (default 0.75)
2. Pattern confidence ≥ 0.8 (must be mature)

`feedback(verdict_id, rating, correction) → feedback_id`

Adjusts linked pattern confidence based on user feedback:

```
def feedback(self, verdict_id, rating, correction=None):
    feedback_id = self.store.save_feedback(verdict_id, "verdict", rating,
correction)
    # Find patterns linked to this verdict
    for pattern in self.store.get_mature_patterns(min_confidence=0.0):
        if verdict_id in pattern.get("source_verdicts", []):
            delta = 0.05 if rating == "positive" else -0.05
            new_conf = clamp(pattern["confidence"] + delta, 0.0, 1.0)
            self.store.update_pattern(pattern["id"], {"confidence": new_conf})
    return feedback_id
```

`stats(domain) → dict`

```
def stats(self, domain=None):
    base_stats = self.store.get_stats(domain)
    base_stats["vector_verdicts"] = self.vectors.verdicts.count()
    base_stats["vector_patterns"] = self.vectors.patterns.count()
    return base_stats
    # Returns: verdicts, patterns, mature_patterns, feedback_entries,
    #          vector_verdicts, vector_patterns, domain
```

---

7. Pattern Lifecycle

Patterns evolve through 4 stages:

```
  Birth              Growth             Maturity           Decay
  ─────              ──────             ────────           ─────
confidence=0.3    confidence grows    confidence≥0.8   confidence drops
hit_count=0       hit_count++         used for         due to negative
new pattern       positive feedback   fast-path        feedback
                  increases conf.     recognition
```

Stage 1: Birth (confidence = 0.3)

A pattern is created when repeated similar verdicts are observed. Initial confidence is low (0.3), too low for fast-path recognition (requires 0.8).

Stage 2: Growth

Each positive feedback on linked verdicts increases confidence by +0.05. Each hit (successful match) increments `hit_count` and updates `last_hit`.

Stage 3: Maturity (confidence ≥ 0.8)

Pattern is now eligible for `recognize()` fast-path matching. When a new query matches a mature pattern, the system can skip full evaluation and return the pattern's expected result.

Stage 4: Decay

Negative feedback decreases confidence by -0.05. A pattern can drop below maturity threshold (0.8) and stop being used for fast-path recognition, though it remains in storage.

Confidence bounds: Always clamped to [0.0, 1.0].

---

8. MCP Server — 5 Tools

Server Identity

```
SERVER_NAME = "furqan-memory"
SERVER_VERSION = "0.1.0"
PROTOCOL_VERSION = "2024-11-05"
```

Tool 1: `furqan_remember`

Store a verdict in local memory.

Input:

```
{
    "question": "Is riba (interest) haram?",
    "verdict": {
        "total_score": 95,
        "gate_results": [],
        "final_judgment": "Prohibited based on Quran and Sunnah"
    },
    "domain": "islamic",
    "tags": ["fiqh", "muamalat"]
}
```

Output:

```
{
    "type": "remembered",
    "verdict_id": "v_abc123def456",
    "message": "Verdict stored successfully. ID: v_abc123def456"
}
```

Tool 2: `furqan_recall`

Search memory for relevant past verdicts using semantic similarity.

Input:

```
{
    "query": "interest and banking",
    "domain": "islamic",
    "limit": 5
}
```

Output:

```
{
    "type": "recall",
    "query": "interest and banking",
    "results": [
        {
            "id": "v_abc123",
            "score": 0.87,
            "question": "Is riba haram?",
            "verdict_data": {"total_score": 95, ...},
            "domain": "islamic",
            "tags": ["fiqh"],
            "created_at": 1711036800.0
```

```
        }
    ],
    "total_found": 1
}
```

Tool 3: `furqan_recognize`

Fast-path pattern matching. Target: <50ms.

Input:

```
{
    "query": "Is bank interest halal?",
    "threshold": 0.75
}
```

Output (matched):

```
{
    "type": "recognized",
    "matched": true,
    "pattern": {
        "id": "p_xyz789",
        "rule": "Riba prohibition pattern",
        "confidence": 0.92,
        "expected_score_min": 0,
        "expected_score_max": 30
    },
    "similarity": 0.89,
    "latency_ms": 12.3
}
```

Output (no match):

```
{
    "type": "recognized",
    "matched": false,
    "message": "No matching pattern found. Proceed with full evaluation.",
    "latency_ms": 8.5
}
```

Tool 4: `furqan_feedback`

Rate a verdict to improve pattern accuracy over time.

Input:

```
{
    "verdict_id": "v_abc123",
    "rating": "positive",
```

```
    "correction": null
}
```

Output:

```
{
    "type": "feedback",
    "feedback_id": "f_def456",
    "message": "Feedback recorded. Thank you for helping improve accuracy."
}
```

Tool 5: `furqan_memory_stats`

Memory usage statistics.

Input:

```
{
    "domain": "islamic"
}
```

Output:

```
{
    "type": "stats",
    "verdicts": 42,
    "patterns": 8,
    "mature_patterns": 3,
    "feedback_entries": 15,
    "vector_verdicts": 42,
    "vector_patterns": 8,
    "domain": "islamic"
}
```

---

9. Installation Guide

For OpenClaw

```
{
    "mcpServers": {
        "furqan-memory": {
            "command": "python",
            "args": ["-m", "furqan_memory.mcp_server"],
            "cwd": "/path/to/al-furqan/furqan-memory"
        }
    }
}
```

For Claude Code / Cursor

Same MCP configuration format — point to `furqan_memory.mcp_server`.

Environment Variables

Variable	Default	Description
`FURQAN_MEMORY_DB`	`furqan_memory.db`	Path to SQLite database file
`FURQAN_MEMORY_VECTORS`	`None` (ephemeral)	Path for ChromaDB persistence

Manual CLI

```
cd /path/to/al-furqan/furqan-memory
python -m furqan_memory.mcp_server
```

---

10. Privacy Guarantees

Guarantee	Implementation
All data local	SQLite file + ChromaDB directory on user's device
No network calls	Zero HTTP requests in the entire codebase
No telemetry	No analytics, no tracking, no error reporting
User-controlled deletion	Delete the DB file = all data gone
No cloud sync	Explicitly not implemented
Open source	Code is auditable

---

11. Performance Targets

Operation	Target	Notes
`recognize()`	<50ms	Fast-path pattern matching
`remember()`	<100ms	SQLite insert + ChromaDB upsert
`recall()`	<200ms	Vector search + SQLite enrichment
`feedback()`	<50ms	SQLite insert + pattern update
`stats()`	<10ms	COUNT queries

---

12. SKILL.md Reference

```
---
name: furqan-memory
description: >
  Client-side memory and learning skill for AI agents.
  Stores verdicts, learns patterns, enables offline reasoning.
  All data stays on user's device — zero network dependency.
version: 0.1.0
transport: stdio
protocol: json-rpc-2.0
---
```

Tools: furqan_remember, furqan_recall, furqan_recognize, furqan_feedback, furqan_memory_stats

---

13. Test Coverage

Test File	Count	Coverage
`tests/test_mcp_memory.py`	16	MCP server protocol + all 5 tools
`tests/test_memory_manager.py`	14	Remember, recall, recognize, feedback, stats
`tests/test_sqlite_store.py`	18	All 4 tables CRUD + search + stats
`tests/test_vector_search.py`	8	Vector add, search, metadata safety
Total	**56**	

Furqan Memory Documentation — March 21, 2026
Variiance R&D — The Criterion Project